

Лабораторная работа №9. Основы работы с системой управления версиями Git

Цель: изучение системы контроля версий Git.

ЗАДАНИЕ

1. Изучите главу 11 теоретического курса «Системы управления версиями». При необходимости установите пакет git.

```
root@Guzovski:/home/guz# sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-el git-email git-gui gitk
  gitweb git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 6 not upgraded.
Need to get 7,430 kB of archives.
After this operation, 38.1 MB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bullseye/main amd64 liberror-perl all 0.1702
-1 [31.0 kB]
```

2. Выполните на рабочем компьютере все практические примеры из данной главы.
3. Создайте в рабочем каталоге пустой репозиторий Git. Настройте с помощью команд, приведенных ниже, осмысленное имя пользователя и адрес электронной почты:

```
git config --global user.name "Your Name"
```

```
git config --global user.email you@example.com
```

```
guz@Guzovski:~/lab9$ git config --global user.name "User"
guz@Guzovski:~/lab9$ git config --global user.email user@example.com
```

4. Скопируйте в рабочий каталог файлы исходного кода, созданные при выполнении лабораторной работы №6, и Make-файл, созданный при выполнении лабораторной работы №7. Укажите, что они включаются в репозиторий. Выполните коммит с пояснением «Начальная версия проекта».

```
guz@Guzovski:~/lab9$ ls
list.c list.h main.c matrix.c matrix.h
guz@Guzovski:~/lab9$
```

5. Внесите осмысленные изменения в файл исходного кода list.c и в Make-файл. Выполните сборку проекта. Изучите состояние файлов в рабочем каталоге командой git status. Добавьте измененные файлы к включению в очередной коммит. Выполните коммит с пояснением «Вторая версия проекта».

```
guz@Guzovski:~/lab9$ git add main.c list.c list.h matrix.c matrix.h Makefile
guz@Guzovski:~/lab9$ git commit -m "Начальная версия проекта"
```

```
[master (root-commit) 958c1b5] Начальная версия проекта
```

```
6 files changed, 104 insertions(+)
```

```
create mode 100644 Makefile
```

```
create mode 100644 list.c
```

```
create mode 100644 list.h
```

```
create mode 100644 main.c
```

```
create mode 100644 matrix.c
```

```
create mode 100644 matrix.h
```

```
guz@Guzovski:~/lab9$ git status
```

```
git log --oneline
```

```
On branch master
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
list.o
```

```
main.o
```

```
matrix.o
```

```
myprog
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

```
958c1b5 (HEAD -> master) Начальная версия проекта
```

6. Создайте пользователя RemoteUser и войдите от имени данного пользователя. Выполните клонирование репозитория в подкаталог рабочего каталога пользователя RemoteUser. Задайте для данного пользователя осмысленное имя и адрес электронной почты (в системе Git).

```
guz@Guzovski: ~/lab9
GNU nano 5.4 list.c *
printf("\n");
}

void DestroyList() {
    struct ListItem *Item, *Item2;
    printf(" Удаление списка.\n");
    Item = Root;
    while (Item != NULL) {
        Item2 = Item->Next;
        free(Item);
        Item = Item2;
    }
    Root = NULL;
}

void ListFunction() {
    printf("Демонстрация работы со списком.\n");
    CreateList();
    ShowList();
    DestroyList();
    printf("\n");
}
```

```
guz@Guzovski: ~/lab9
GNU nano 5.4 Makefile *
CC := gcc
CFLAGS := -Wall -O2
INST_DIR ?= /home/user/test_for_make

all: myprog

myprog: main.o list.o matrix.o
    $(CC) $(CFLAGS) -o $@ $^ -lm

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

install: myprog
    mkdir -p $(INST_DIR)
    cp myprog $(INST_DIR)

clean:
    rm -f *.o myprog

run: myprog
    ./myprog
```

```

guz@Guzovski:~/lab9$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Makefile
        modified:   list.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        list.o
        main.o
        matrix.o
        myprog

no changes added to commit (use "git add" and/or "git commit -a")
guz@Guzovski:~/lab9$ git add list.c Makefile
guz@Guzovski:~/lab9$ git commit -m "Вторая версия проекта"
[master 0cc6224] Вторая версия проекта
 2 files changed, 5 insertions(+)
guz@Guzovski:~/lab9$ git log --oneline
0cc6224 (HEAD -> master) Вторая версия проекта
958c1b5 Начальная версия проекта
guz@Guzovski:~/lab9$

```

7. Внесите осмысленные изменения в файл исходного кода list.c, main.c и в Make- файл. Выполните сборку проекта. Добавьте измененные файлы к включению в очередной коммит. Добавьте к проекту файл скрипта на языке Shell с несколькими осмысленными операторами и именем test.sh. Добавьте его для включения в коммит.

```

root@Guzovski:/home/guz/lab9# sudo adduser remoteuser
Adding user `remoteuser' ...
Adding new group `remoteuser' (1001) ...
Adding new user `remoteuser' (1001) with group `remoteuser' ...
Creating home directory `/home/remoteuser' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N] n
Changing the user information for remoteuser
Enter the new value, or press ENTER for the default
    Full Name []: RemoteUser
    Room Number []: 1
    Work Phone []: 1
    Home Phone []: 1
    Other []: 1
Is the information correct? [Y/n] y
root@Guzovski:/home/guz/lab9#

```

```
root@Guzovski:/home/guz/lab9# git config --global user.name "RemoteUser"
root@Guzovski:/home/guz/lab9# git config --global user.email remoteuser@example.com
root@Guzovski:/home/guz/lab9#
```

```
root@Guzovski:/home/guz/lab9# cat > test.sh << 'EOF'
#!/bin/bash
echo "Запуск тестового скрипта..."
echo "Компиляция и запуск программы:"
make run
echo "Проверка завершена."
EOF
chmod +x test.sh
guz@Guzovski:~/lab9$ git add list.c main.c Makefile test.sh
guz@Guzovski:~/lab9$ git commit -m "Версия от RemoteUser"
[master fbcf2d0] Версия от RemoteUser
4 files changed, 25 insertions(+)
create mode 100755 test.sh
guz@Guzovski:~/lab9$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .swp
        list.o
        main.o
        matrix.o
        myprog

nothing added to commit but untracked files present (use "git add" to track)
guz@Guzovski:~/lab9$ git log --oneline
fbcf2d0 (HEAD -> master) Версия от RemoteUser
0cc6224 Вторая версия проекта
958c1b5 Начальная версия проекта
guz@Guzovski:~/lab9$
```

8. Зайдите от имени основного пользователя User. Просмотрите историю версий проекта командой git log. Проверьте текущее содержимое файлов исходного репозитория.

```

guz@Guzovski:~/lab9$ git log --oneline
fbcf2d0 (HEAD -> master) Версия от RemoteUser
0cc6224 Вторая версия проекта
958c1b5 Начальная версия проекта
-----
guz@Guzovski:~/lab9$ ls -l
total 64
-rw-r--r-- 1 guz  guz   1389 Dec 12 22:36 list.c
-rw-r--r-- 1 guz  guz    21 Dec 12 22:17 list.h
-rw-r--r-- 1 root root 4272 Dec 12 22:40 list.o
-rw-r--r-- 1 guz  guz   311 Dec 12 22:37 main.c
-rw-r--r-- 1 root root 1992 Dec 12 22:40 main.o
-rw-r--r-- 1 guz  guz   344 Dec 12 22:38 Makefile
-rw-r--r-- 1 guz  guz   279 Dec 12 22:17 matrix.c
-rw-r--r-- 1 guz  guz    23 Dec 12 22:17 matrix.h
-rw-r--r-- 1 root root 2064 Dec 12 22:40 matrix.o
-rwxr-xr-x 1 root root 17224 Dec 12 22:40 myprog
-rwxr-xr-x 1 root root  186 Dec 12 22:39 test.sh
guz@Guzovski:~/lab9$
guz@Guzovski:~/lab9$ ./test.sh
Запуск тестового скрипта...
Компиляция и запуск программы:
./myprog
Пример программы на языке C

Демонстрация работы со списком.
Создание списка.
Просмотр списка.
(1), (2), (3), (4), (5), Сумма элементов: 15

Удаление списка.

Запущен пример с матрицей.
Программа завершена успешно.
Проверка завершена.  _

```

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего предназначены системы управления версиями?

Системы управления версиями предназначены для хранения, отслеживания и управления изменениями в исходном коде и других файлах проекта.

Чем отличаются централизованные и распределенные системы управления версиями? =

Централизованные системы используют единый сервер как точку хранения и управления версиями, а распределенные системы позволяют каждому пользователю иметь полную копию репозитория, обеспечивая автономную работу и гибкость.

Какие основные функции выполняет система управления версиями? = Основные функции включают хранение истории изменений, отслеживание версий файлов, управление параллельной работой нескольких пользователей, разрешение конфликтов и восстановление предыдущих состояний проекта.

2. Поясните смысл терминов «update», «checkout», «commit», «merge» в контексте систем управления версиями.

Update — получение последних изменений из репозитория. = **Checkout** — переключение на определённую версию или ветку проекта. = **Commit** — фиксация изменений в локальном репозитории с созданием новой версии. = **Merge** — объединение изменений из разных веток или версий в одну.

3. В каком состоянии может находиться файл, включенный в проект в системе управления версиями Git?

Файл может находиться в состояниях: неотслеживаемый (untracked), отслеживаемый (tracked), изменённый (modified), подготовленный к фиксации (staged), зафиксированный (committed).

4. Как производится переход из одного состояния файла в другое состояние в системе управления версиями Git?

Переход между состояниями файла осуществляется с помощью команд Git:

- Из состояния **untracked** в **staged** — команда `git add`.
- Из состояния **modified** в **staged** — команда `git add`.
- Из состояния **staged** в **committed** — команда `git commit`.
- Из состояния **committed** в **modified** — изменение содержимого файла в рабочем каталоге.
- Из состояния **committed** в **staged** — повторное использование `git add` после изменений.

Таким образом, переходы управляются комбинацией редактирования файлов и выполнения соответствующих команд Git.